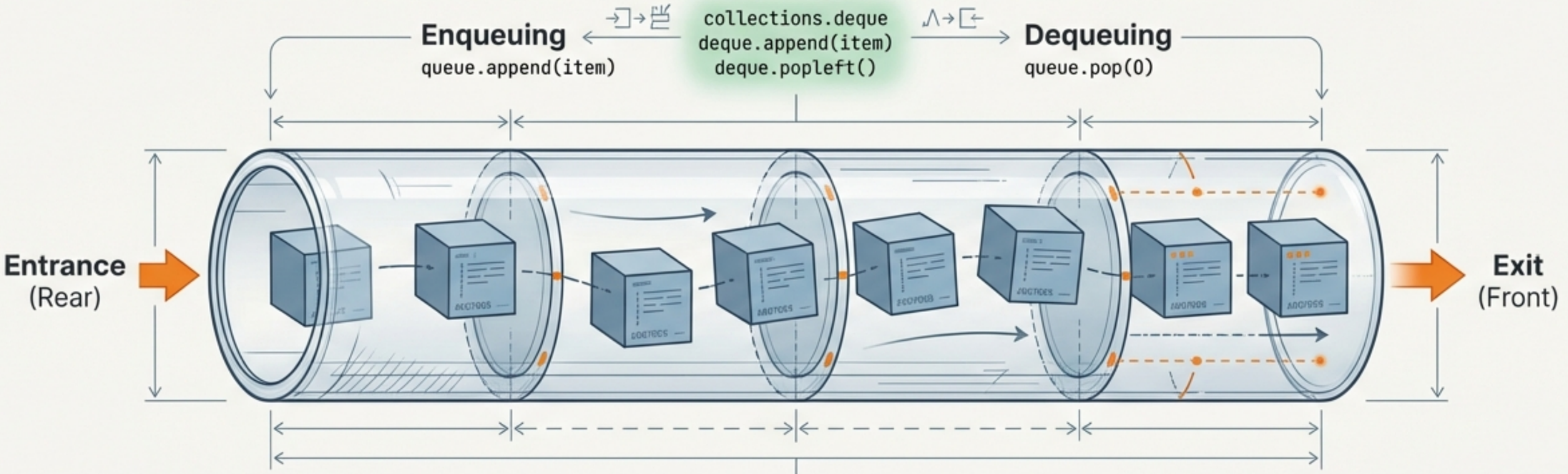
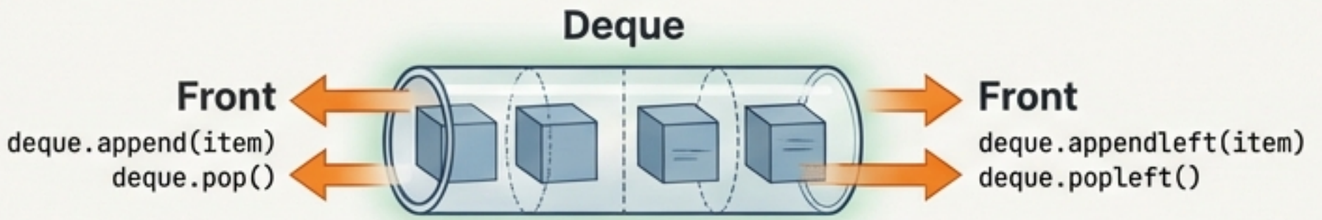
Queue & Deque in Python

From Algorithmic Logic to Implementation



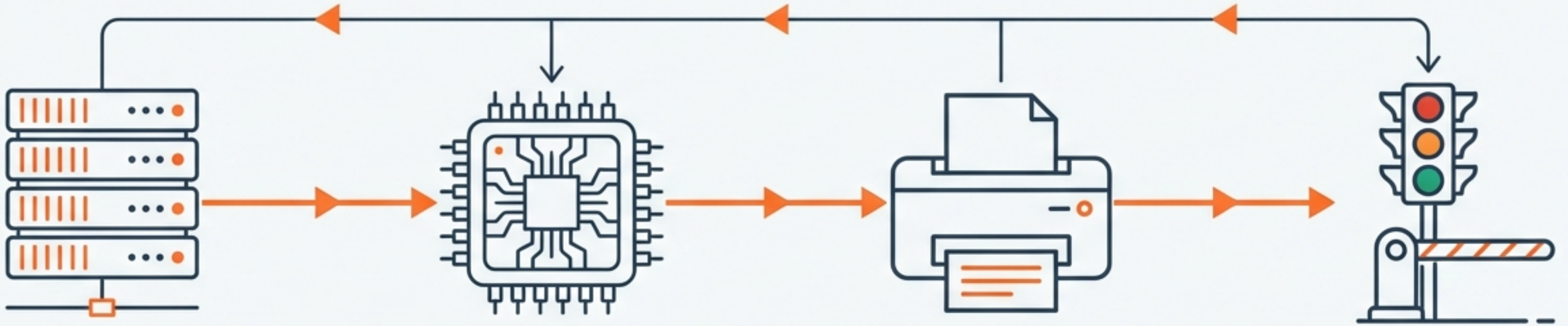


Data Structures & Algorithms Series

NotebookLM

The Invisible Backbone of Modern Software

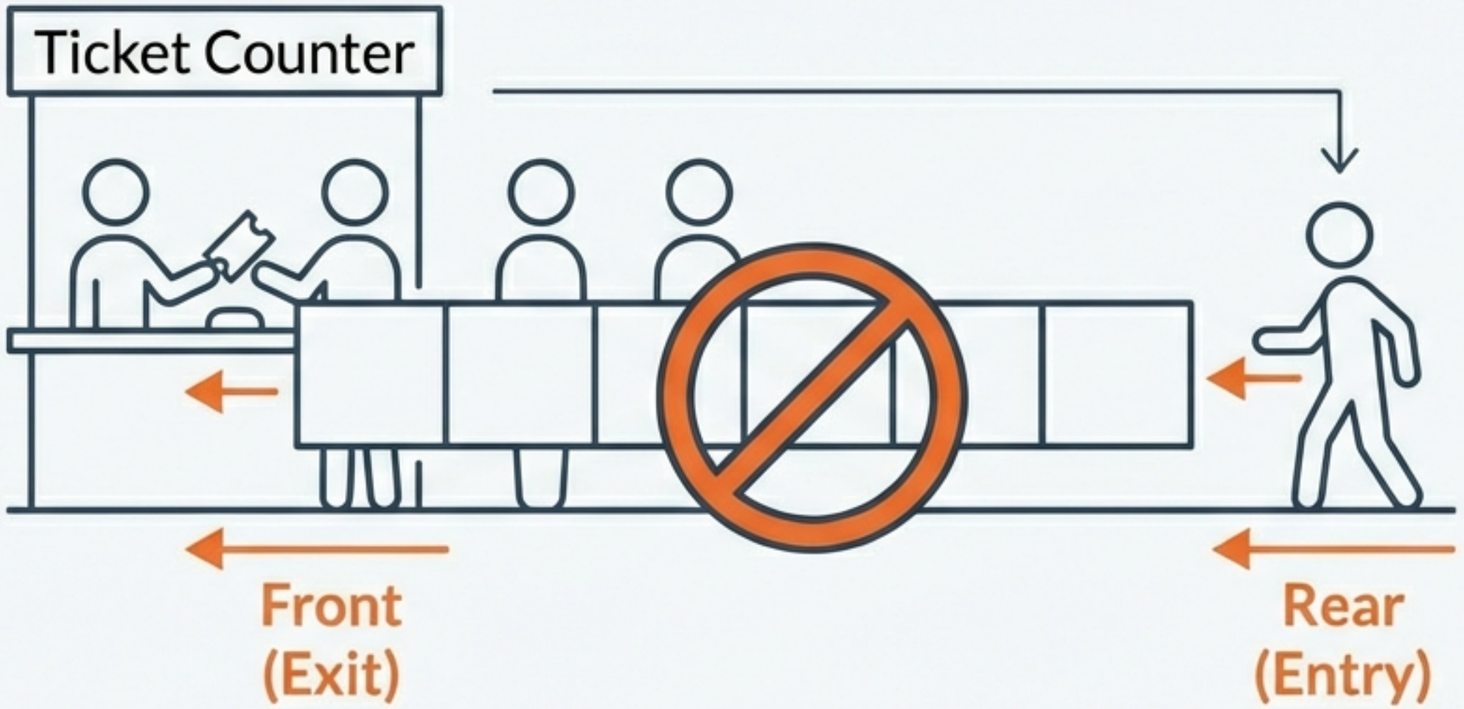
Queues aren’t just for interviews; they manage the flow of the digital world.



Whether it’s a cron job or a complex minmax window problem, the underlying logic is the Queue.

NotebookLM

The Rule of Entry: First In, First Out (FIFO)



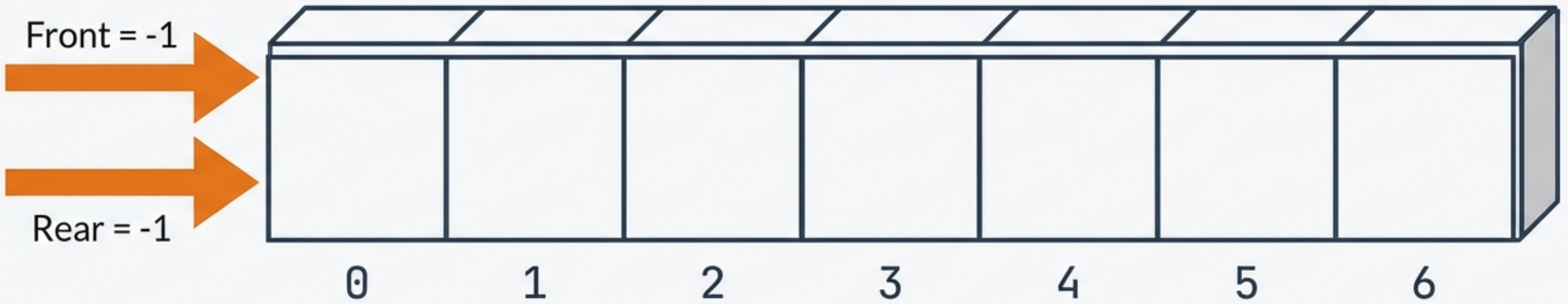
- FIFO: The element inserted first is the first to be removed.

  Restriction: strictly no access to middle elements.

NotebookLM

The Low-Level Challenge: Manual Memory Management

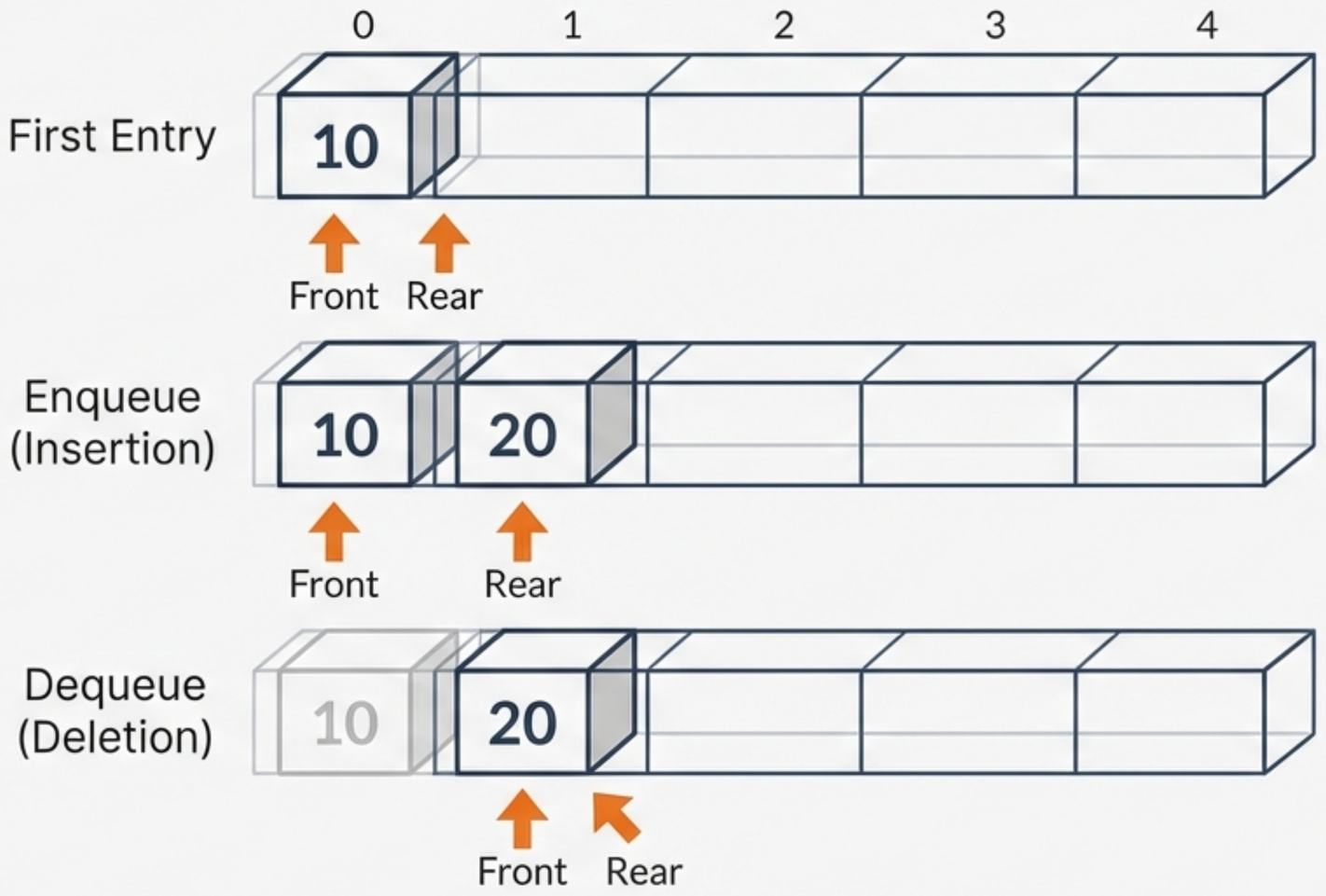
In languages like C or C++, arrays have a fixed size. We must manually track positions using pointers.



Constraint: You must constantly check for Overflow (Full) and Underflow (Empty).

NotebookLM

Mechanics of Fixed-Array Operations

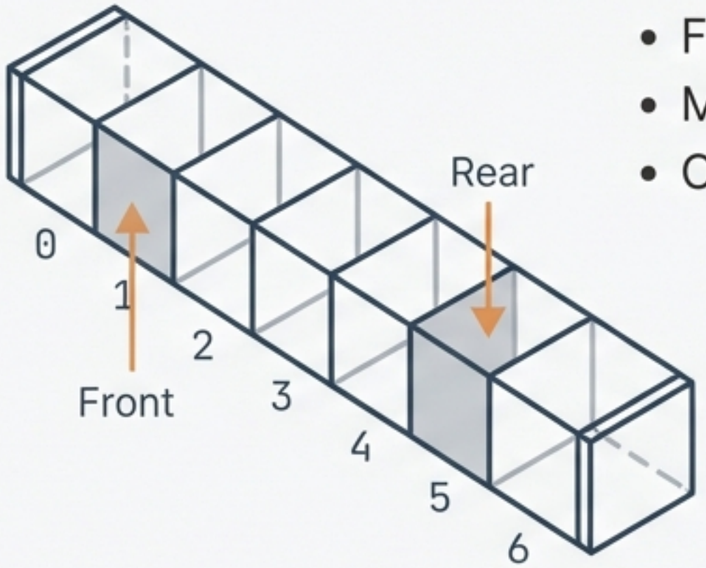


Data stays put. Pointers move.

NotebookLM

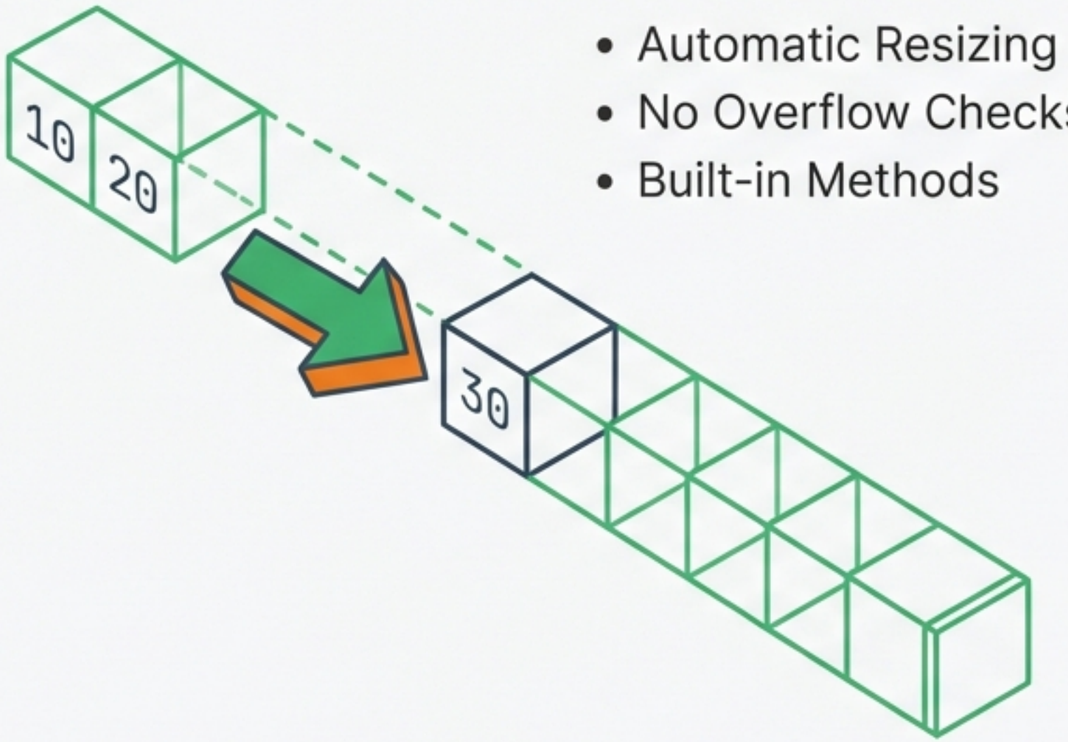
The Python Abstraction: Dynamic Lists

The Old Way



- Fixed Size
- Manual Pointers
- Overflow Risks

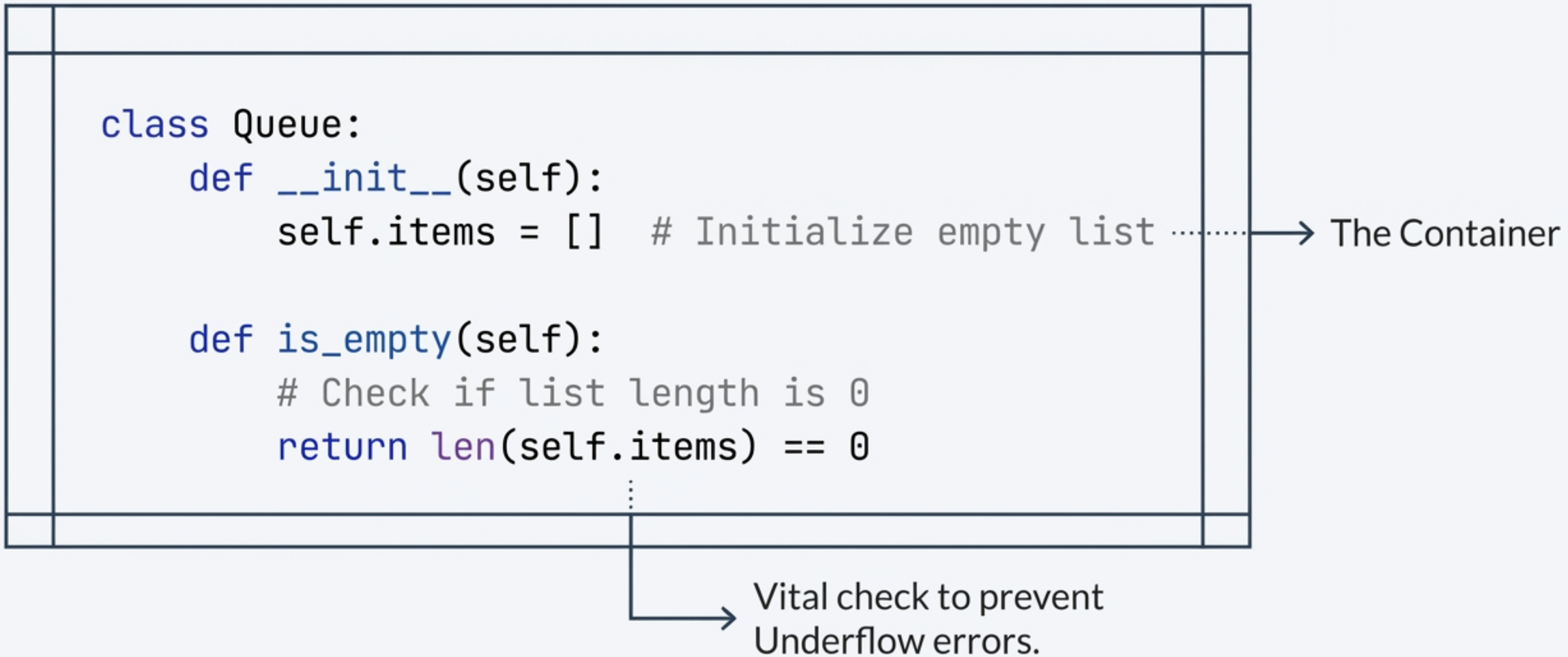
The Python Way



- Automatic Resizing
- No Overflow Checks
- Built-in Methods

NotebookLM

Implementing the Queue Structure

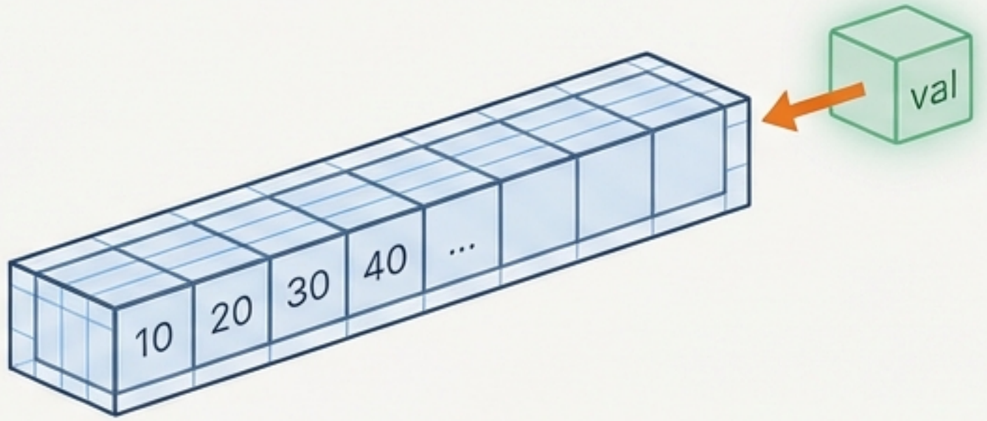


NotebookLM

Enqueue and Dequeue in Python

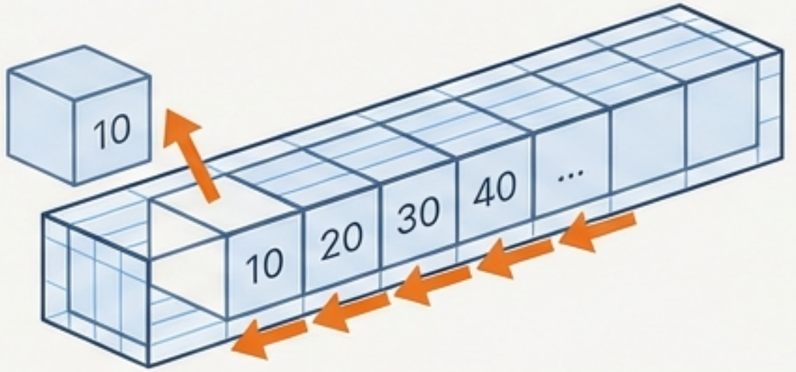
Enqueue: insert(val)

```
self.items.append(val)
```



Dequeue: delete()

```
self.items.pop(0)
```



⚠ **Note:** In Python, data moves. Shifting all elements costs O(n).

NotebookLM

Trace: The Queue Lifecycle

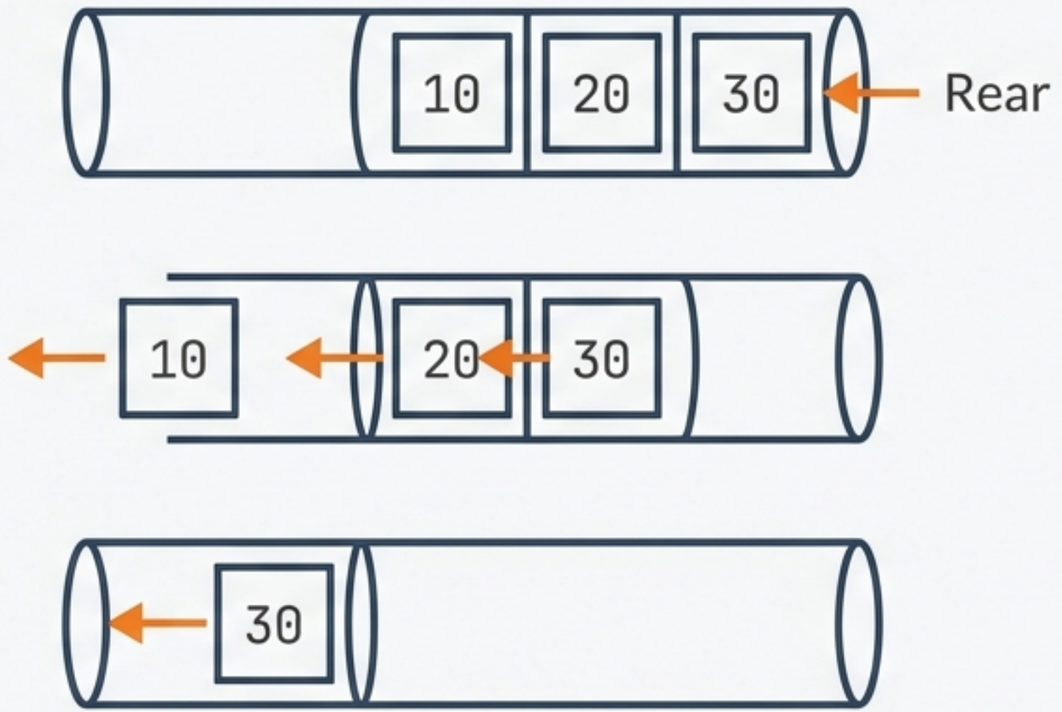
Code Steps

Step 1: JetBrains Mono
`q.insert(10), q.insert(20), q.insert(30)`

Step 2: JetBrains Mono
`q.delete() (Removes 10)`

Step 3: JetBrains Mono
`q.delete() (Removes 20)`

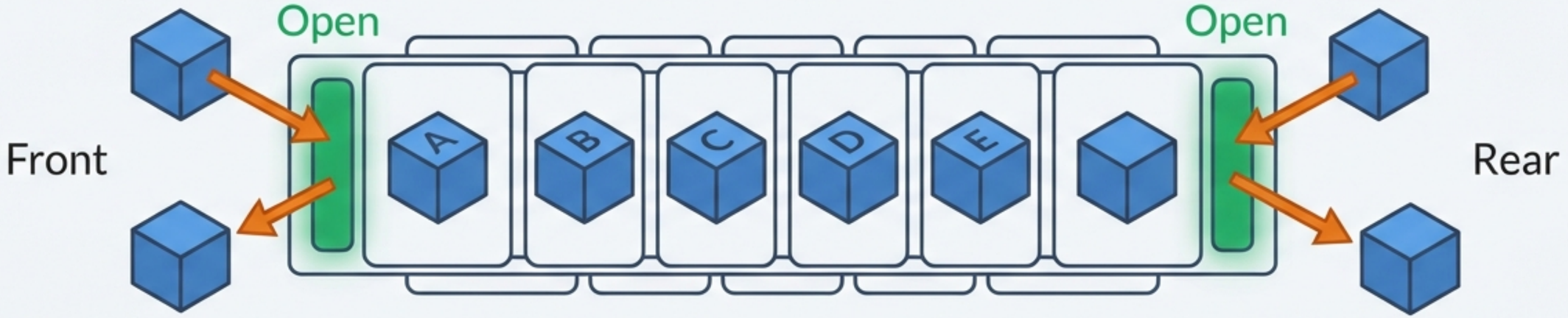
Visual State



NotebookLM

Breaking the Rules: The Deque

Double-Ended Queue

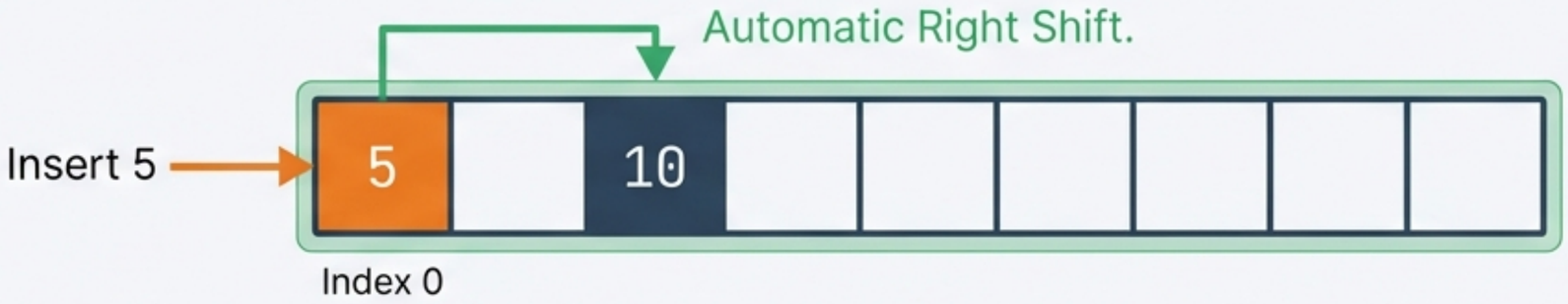


Definition: A linear data structure supporting insertion and deletion from both ends.

Middle access is still restricted.

NotebookLM

The Logic Gap: Insertion at Front

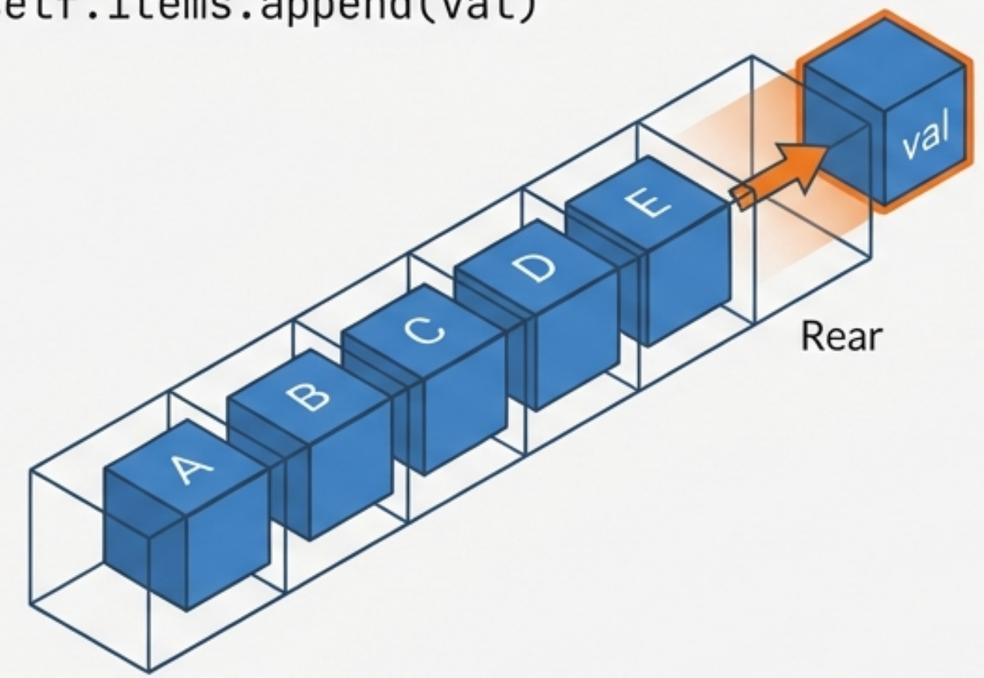
The Problem	
	Low-Level: If Front is at 0, you cannot move left. No space.
The Python Solution	
	Python: Automatic Right Shift creates space at Index 0.

NotebookLM

Deque Operations: Insertion

Insert Rear

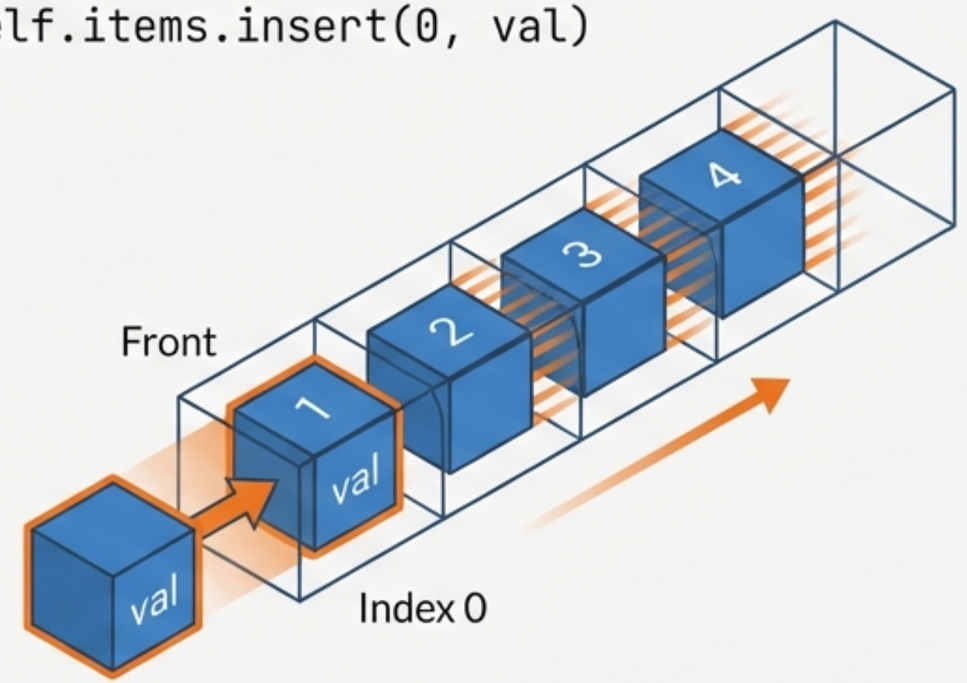
`self.items.append(val)`



Simple block attachment to the tail. Low friction.

Insert Front

`self.items.insert(0, val)`



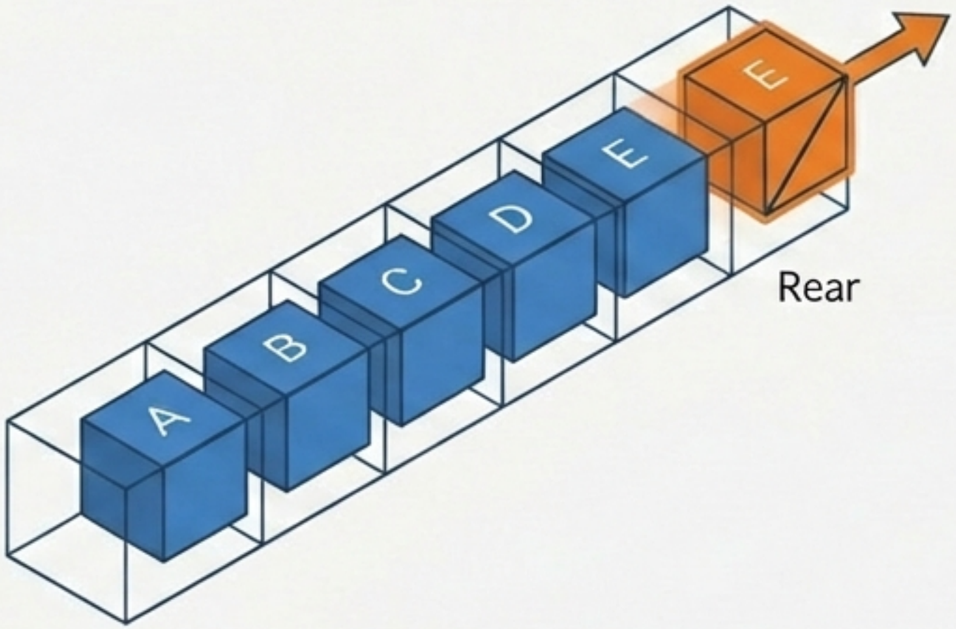
A block forcing its way into Index 0. All other blocks (1, 2, 3...) have small 'motion blur' lines to the right, indicating they are being shoved over. Heavier operation due to shifting.

NotebookLM

Deque Operations: Deletion

Delete Rear

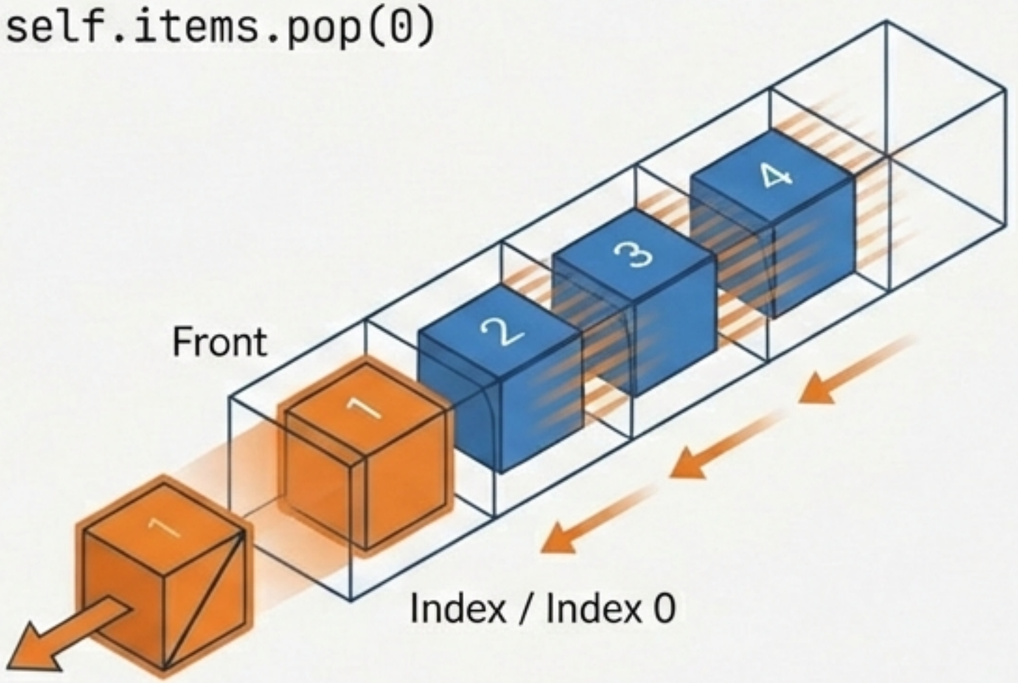
`self.items.pop()`



Simple block removal from the tail. Low friction.

Delete Front

`self.items.pop(0)`

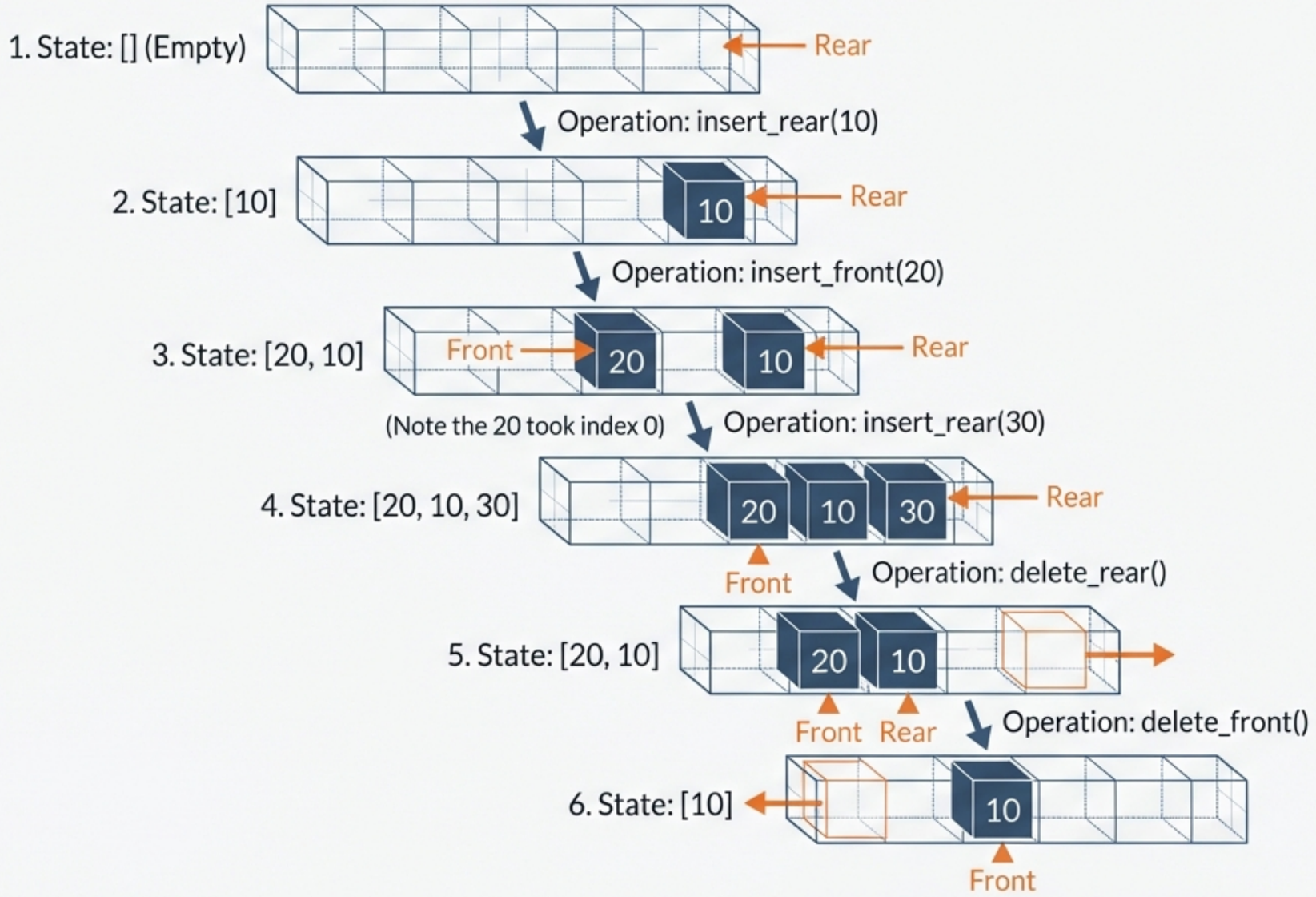


The block at Index 0 is removed. All subsequent blocks have 'motion blur' lines to the LEFT, indicating they are sliding back to fill the gap. Heavier operation due to shifting.

Safety: Always check if empty first.

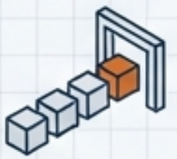
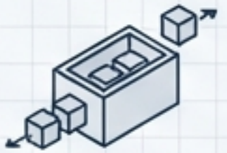
NotebookLM

Full Deque Simulation Trace



NotebookLM

Summary & Key Takeaways

Structure	Principle	Best Use Case
Queue	FIFO	Good for: Ordering Processes 
Deque	Double-Ended	Good for: Complex Buffers 

Python Note: Syntax is easy, but `insert(0)` and `pop(0)` force a full list shift ($O(n)$$).

Next Up: Circular Queues

NotebookLM